

Section 4.3 of Sterbac’s paper in Agda: annotated versus unannotated Russell universes

Formalisation notes ($\sim$ /DOMAIN/ERT)

24–25 September 2026

This note records the systems T_R (Russell universes, annotated λ and **app**) and T_P (unannotated, “pure type system”) as formalised in Agda, how they differ from **sterbac1.pdf** (§4.3, appendices A.3/A.4; the same material is §6 of **sterbac3.pdf**), what is proved, and a gap in the paper’s proof of Lemma 4.17. Everything is in Agda with `--without-K` `--exact-split`, without postulates and without termination pragmas. Lemma 4.18 and Theorem 4.19 are proved *without normalisation* (§6), from Π -injectivity, injectivity of universes and Π/U no-confusion only—the three properties the paper itself assumes (Theorems 7–9 of **sterbac3.pdf**). An earlier proof under weak β -normalisation of T_P (§8) is kept for comparison. The type-in-type version, where normalisation fails, is treated in §7.

Premises printed in grey are *presuppositions*: they are not in the paper’s rules, they are admissible (derivable from the other premises), and they are there only to make the metatheory, the adequacy proof and the section of strip structurally recursive (see §4).

1 Syntax

$$\begin{aligned} T_R : \quad A, B, a, b, c &::= v_i \mid U_l \mid \Pi(A, B) \mid \lambda(A, B, b) \mid \mathbf{app}(A, B, c, a) \\ T_P : \quad A, B, a, b, c &::= v_i \mid U_l \mid \Pi A B \mid \lambda A b \mid c a \end{aligned}$$

de Bruijn indices, contexts $\Gamma ::= () \mid \Gamma.A$. The syntax of T_P is the core syntax of the finite-element model (**Model/Core.agda**). The forgetful map is

$$\text{strip}(v_i) = v_i, \quad \text{strip}(U_l) = U_l, \quad \text{strip}(\Pi(A, B)) = \Pi \text{strip}(A) \text{strip}(B),$$

$$\text{strip}(\lambda(A, B, b)) = \lambda \text{strip}(A) \text{strip}(b), \quad \text{strip}(\mathbf{app}(A, B, c, a)) = \text{strip}(c) \text{strip}(a).$$

It forgets the codomain annotation of λ and both annotations of **app** (this is Sterbac’s **strip**; it keeps less than Streicher’s strip_1 , which keeps λ ’s annotation, and than the erasure $|\cdot|$ of §4.2, which keeps all annotations of **app**). **strip** commutes with renaming, substitution and context lookup.

2 The system T_R (**RussellTyping.agda**)

Judgements: $\vdash \Gamma$, $\Gamma \vdash A$ type, $\Gamma \vdash a : A$, $\Gamma \vdash A = B$, $\Gamma \vdash a = b : A$.

Contexts and types.

$$\frac{}{\vdash ()} \qquad \frac{\Gamma \vdash A \text{ type}}{\vdash \Gamma.A} \qquad \frac{\Gamma \vdash A : U_l}{\Gamma \vdash A \text{ type}}$$

Typing.

$$\begin{aligned} &\frac{\vdash \Gamma}{\Gamma \vdash v_i : \Gamma(i)} \qquad \frac{\Gamma \vdash a : A \quad \Gamma \vdash A = B}{\Gamma \vdash a : B} \qquad \frac{\vdash \Gamma}{\Gamma \vdash U_l : U_{l+1}} \qquad \frac{\Gamma \vdash A : U_l}{\Gamma \vdash A : U_{l+1}} \\ &\frac{\Gamma \vdash A : U_l \quad \Gamma.A \vdash B : U_l}{\Gamma \vdash \Pi(A, B) : U_l} \qquad \frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma.A \vdash b : B}{\Gamma \vdash \lambda(A, B, b) : \Pi(A, B)} \\ &\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash c : \Pi(A, B) \quad \Gamma \vdash a : A}{\Gamma \vdash \mathbf{app}(A, B, c, a) : B[a]} \end{aligned}$$

Type equality.

$$\begin{array}{c}
\frac{\Gamma \vdash A \text{ type}}{\Gamma \vdash A = A} \quad \frac{\Gamma \vdash A = B}{\Gamma \vdash B = A} \quad \frac{\Gamma \vdash A = B \quad \Gamma \vdash B = C}{\Gamma \vdash A = C} \quad \frac{\Gamma \vdash A = B : \mathcal{U}_l}{\Gamma \vdash A = B} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash A = A' \quad \Gamma.A \vdash B = B'}{\Gamma \vdash \Pi(A, B) = \Pi(A', B')}
\end{array}$$

Term equality.

$$\begin{array}{c}
\frac{\Gamma \vdash a : A}{\Gamma \vdash a = a : A} \quad \frac{\Gamma \vdash a = b : A}{\Gamma \vdash b = a : A} \quad \frac{\Gamma \vdash a = b : A \quad \Gamma \vdash b = c : A}{\Gamma \vdash a = c : A} \quad \frac{\Gamma \vdash a = b : A \quad \Gamma \vdash A = B}{\Gamma \vdash a = b : B} \\
\\
\frac{\Gamma \vdash A = B : \mathcal{U}_l}{\Gamma \vdash A = B : \mathcal{U}_{l+1}} \\
\\
\frac{\Gamma \vdash A : \mathcal{U}_l \quad \Gamma.A \vdash B : \mathcal{U}_l \quad \Gamma \vdash A = A' : \mathcal{U}_l \quad \Gamma.A \vdash B = B' : \mathcal{U}_l}{\Gamma \vdash \Pi(A, B) = \Pi(A', B') : \mathcal{U}_l} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma.A \vdash b : B \quad \Gamma.A \vdash b = b' : B}{\Gamma \vdash \lambda(A, B, b) = \lambda(A, B, b') : \Pi(A, B)} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash A = A' \quad \Gamma.A \vdash B = B' \quad \Gamma.A \vdash b : B}{\Gamma \vdash \lambda(A, B, b) = \lambda(A', B', b) : \Pi(A, B)} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash c = c' : \Pi(A, B) \quad \Gamma \vdash a : A}{\Gamma \vdash \mathbf{app}(A, B, c, a) = \mathbf{app}(A, B, c', a) : B[a]} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash c : \Pi(A, B) \quad \Gamma \vdash a = a' : A \quad \Gamma \vdash B[a] = B[a']}{\Gamma \vdash \mathbf{app}(A, B, c, a) = \mathbf{app}(A, B, c, a') : B[a]} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash A = A' \quad \Gamma.A \vdash B = B' \quad \Gamma \vdash c : \Pi(A, B) \quad \Gamma \vdash a : A}{\Gamma \vdash \mathbf{app}(A, B, c, a) = \mathbf{app}(A', B', c, a) : B[a]} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma.A \vdash b : B \quad \Gamma \vdash a : A}{\Gamma \vdash \mathbf{app}(A, B, \lambda(A, B, b), a) = b[a] : B[a]} (\beta) \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash c : \Pi(A, B)}{\Gamma \vdash c = \lambda(A, B, \mathbf{app}(A \uparrow, B \uparrow^+, c \uparrow, v_0)) : \Pi(A, B)} (\eta)
\end{array}$$

3 The system T_P (PTyping.agda)

The rules of T_P are those of T_R with the annotations removed: contexts, types, type equality and the structural rules are as above (on T_P syntax), and

$$\begin{array}{c}
\frac{\Gamma \vdash A : \mathbb{U}_l \quad \Gamma.A \vdash B : \mathbb{U}_l}{\Gamma \vdash \Pi A B : \mathbb{U}_l} \quad \frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma.A \vdash b : B}{\Gamma \vdash \lambda A b : \Pi A B} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash c : \Pi A B \quad \Gamma \vdash a : A}{\Gamma \vdash c a : B[a]} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash A = A' \quad \Gamma.A \vdash B = B'}{\Gamma \vdash \Pi A B = \Pi A' B'} \\
\\
\frac{\Gamma \vdash A : \mathbb{U}_l \quad \Gamma.A \vdash B : \mathbb{U}_l \quad \Gamma \vdash A = A' : \mathbb{U}_l \quad \Gamma.A \vdash B = B' : \mathbb{U}_l}{\Gamma \vdash \Pi A B = \Pi A' B' : \mathbb{U}_l} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma.A \vdash b : B \quad \Gamma.A \vdash b = b' : B}{\Gamma \vdash \lambda A b = \lambda A b' : \Pi A B} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash A = A' \quad \Gamma.A \vdash b : B}{\Gamma \vdash \lambda A b = \lambda A' b : \Pi A B} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash c = c' : \Pi A B \quad \Gamma \vdash a : A}{\Gamma \vdash c a = c' a : B[a]} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash c : \Pi A B \quad \Gamma \vdash a = a' : A}{\Gamma \vdash c a = c a' : B[a]} \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma.A \vdash b : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda A b) a = b[a] : B[a]} (\beta) \\
\\
\frac{\Gamma \vdash A \text{ type} \quad \Gamma.A \vdash B \text{ type} \quad \Gamma \vdash c : \Pi A B}{\Gamma \vdash c = \lambda A (c \uparrow v_0) : \Pi A B} (\eta)
\end{array}$$

There is no counterpart of the T_R rule changing the annotations of an application: its two sides have the same stripping.

4 Differences with the paper

1. **Presupposition premises** (grey). The paper's rules have none. In T_R they are needed at two places:
 - the adequacy proof of the domain model (for Π -injectivity) is a structural induction whose congruence cases need the *two-substitution* validity of the codomain / body, which must come from a subderivation (this is the same obstruction as MIN's **conv-Pi**);
 - Theorem 4.15 and Theorem 4.19 are structural inductions which need the typings of the components of binders (to lift the extended context).

The paper's rules are admissible in T_R : they are the lemmas **mk-conv-Ty-Pi**, **mk-conv-cong-Pi**, **mk-conv-cong-Lam-Ty**, **mk-conv-cong-App-Ty** (**RussellMeta.agda**), using the presupposition lemmas. For T_P the corresponding admissibility is not formalised (it needs presupposition lemmas for T_P).

2. **Universes and cumulativity** are one step at a time ($\mathbb{U}_l : \mathbb{U}_{l+1}$, $A : \mathbb{U}_l \Rightarrow A : \mathbb{U}_{l+1}$); the paper's $\mathbb{U}_m : \mathbb{U}_n$ ($m < n$) and $A : \mathbb{U}_n \Rightarrow A : \mathbb{U}_p$ ($n < p$) are derivable (**cum-1e**). $\Gamma \vdash \mathbb{U}_n \text{ type}$ is derived.
3. **Variables** are typed by lookup ($\Gamma \vdash v_i : \Gamma(i)$ with the weakened type) instead of the paper's two weakening rules.
4. The congruences of λ and **app** are split into one rule per argument (body / annotation, function / argument / annotations).

5 What is proved without any hypothesis

Theorem 1 (Adequacy of the domain model, `Adeq/Driver.agda`). *The finite-element (Coquand–Huber) model with levelled universe codes is adequate for T_R : every derivation is valid for the rank-stratified logical relation, by structural induction.*

Theorem 2 (`PiInjectivityR.agda`, `RussellInversion.agda`). *In T_R : if $\Gamma \vdash \Pi(C_0, B_0) = \Pi(C_1, B_1)$ then $\Gamma \vdash C_0 = C_1$ and $\Gamma.C_0 \vdash B_0 = B_1$ (Π -injectivity, from adequacy); if $\Gamma \vdash \mathbb{U}_m = \mathbb{U}_k$ then $m = k$; and $\Gamma \vdash \Pi(A, B) = \mathbb{U}_m$ never holds (from soundness of the model).*

Theorem 3 (4.15, `StripDeriv.agda`). *If $\Gamma \vdash J$ in T_R then $\text{strip}(\Gamma) \vdash \text{strip}(J)$ in T_P .*

Theorem 4 (Subject reduction, `SubjectReductionR.agda`). *Let $u \rightarrow u'$ be a β -step of T_R at a position kept by strip ($\text{app}(A, B, \lambda(A', B', b), a) \rightarrow b[a]$, annotations arbitrary). If $\Gamma \vdash u : T$ then $\Gamma \vdash u' : T$. (The β case uses Π -injectivity.)*

Lemma 5 (Lifting of reductions, `RussellStep.agda`). *If $\text{strip}(u) \rightarrow M'$ is a β -step of T_P , then $u \rightarrow u'$ in T_R with $\text{strip}(u') = M'$, for every u .*

Lemma 6 (Normal forms, `StripUniq.agda`: `nf-conv`). *Let $\Gamma \vdash u_0 : A_0$, $\Gamma \vdash u_1 : A_1$ in T_R with $\text{strip}(u_0) = \text{strip}(u_1)$ a β -normal form, and $\Gamma \vdash A_0 = A_1$. Then $\Gamma \vdash u_0 = u_1 : A_0$. Similarly two types with the same β -normal stripping are equal.*

The proof is by induction on the normal form, together with: for *neutral* strippings, u_0, u_1 are equal at a common type P lying below both A_0 and A_1 , where “ P below T ” means $\Gamma \vdash P = T$, or $\Gamma \vdash P = \mathbb{U}_k$ and $\Gamma \vdash T = \mathbb{U}_m$ with $k \leq m$ (`SubTy`). In a neutral application the annotations are forced by the type of the head; in a λ by its type (via Π -injectivity); cumulativity only acts at the root, where it changes no syntax.

6 Lemma 4.18 without normalisation

`StripUniqNF.agda`; the only non-syntactic ingredients are Π -injectivity, injectivity of universes and $\Pi/\mathbb{U}$ no-confusion. The rest is a structural induction on the term, i.e. a primitive recursive transformation of derivations.

What the induction hypothesis must say. In the counterexample of §9, $f_0 = \lambda(\mathbb{U}_1, \mathbb{U}_1, v_0)$ and $f_1 = \lambda(\mathbb{U}_1, \mathbb{U}_2, v_0)$ have the same stripping but are *not* convertible: their types $\Pi(\mathbb{U}_1, \mathbb{U}_1)$, $\Pi(\mathbb{U}_1, \mathbb{U}_2)$ differ. So the hypothesis has to relate terms whose types differ by cumulativity *under* Π . T_R has no such subtyping, but it is definable by η -expansion:

$$\text{coe}_{\Pi(A, B) \leq \Pi(A, L)}(t) = \lambda(A, L, \text{coe}_{B \leq L}(\text{app}(A \uparrow, B \uparrow^+, t \uparrow, v_0))), \quad \text{coe}_{\mathbb{U}_k \leq \mathbb{U}_m}(t) = t.$$

Joins. $\text{Join}_\Gamma(T_0, T_1, L)$ is generated by

- $\Gamma \vdash T_0 = L$ and $\Gamma \vdash T_1 = L$ (coercions: identity);
- $\Gamma \vdash T_0 = \mathbb{U}_k$, $\Gamma \vdash T_1 = \mathbb{U}_m$, $L = \mathbb{U}_{\max(k, m)}$ (identity; this is where cumulativity enters);
- $\Gamma \vdash T_i = \Pi(A, B_i)$ and $\text{Join}_{\Gamma.A}(B_0, B_1, L')$, $L = \Pi(A, L')$ (coercions as above).

A join carries its coercions $\text{coe}_0, \text{coe}_1$. They are well typed ($t : T_i \Rightarrow \text{coe}_i(t) : L$), congruent, stable under substitution (syntactically: $\text{coe}_k(t)[\sigma] = \text{coe}_{k[\sigma]}(t[\sigma])$), and compute under application: $\text{app}(A, L, \text{coe}_{\Pi(A, B) \leq \Pi(A, L)}(t), a) = \text{coe}_{B[a] \leq L[a]}(\text{app}(A, B, t, a))$ by β .

Lemma 7 (main). *If $\Gamma \vdash u_0 : T_0$, $\Gamma \vdash u_1 : T_1$ and $\text{strip}(u_0) = \text{strip}(u_1)$, then there is a join $\text{Join}_\Gamma(T_0, T_1, L)$ with $\Gamma \vdash \text{coe}_0(u_0) = \text{coe}_1(u_1) : L$.*

The proof is by induction on u_0 , with T_0, T_1 arbitrary derivable types. Inversion gives the principal type P_i of u_i with $P_i \leq T_i$ (conversion, or universes $\mathbb{U}_k \leq \mathbb{U}_m$). A join of P_0, P_1 is then moved to one of T_0, T_1 : by transport along conversions, or, if a universe is involved, by a universe join of larger level.

- $v_i, \mathbb{U}_l$: trivial. $\Pi(A_i, B_i)$: the induction hypothesis on the components gives joins of universes, hence $A_0 = A_1 : \mathbb{U}_m$ and $B_0 = B_1 : \mathbb{U}_m$.
- $\lambda(A_i, B_i, b_i)$: $A_0 = A_1$ as above; the induction hypothesis on the bodies, at their annotated types B_0, B_1 , gives $\text{Join}_{\Gamma.A_0}(B_0, B_1, L')$. The Π -join $\text{Join}(\Pi(A_0, B_0), \Pi(A_1, B_1), \Pi(A_0, L'))$ works because $\text{coe}(\lambda(A, B_0, b_0)) = \lambda(A, L', \text{coe}(b_0))$ by β under the binder.

- $\text{app}(C_i, D_i, c_i, a_i)$: the induction hypothesis on c_0, c_1 at $\Pi(C_i, D_i)$ gives a join which, by no-confusion and Π -injectivity, is either a conversion or a Π -join $\Pi(A, L')$ with $C_i = A, D_i = B_i$. The induction hypothesis on a_0, a_1 together with the diagonal lemma below gives $a_0 = a_1 : C_0$. Applying the coercion equation to a_0 and using the β -law of coercions gives $\text{coe}_{B_0[a_0] \leq L'[a_0]}(u_0) = \text{coe}_{B_1[a_0] \leq L'[a_0]}(u_1) : L'[a_0]$, with the substituted join $\text{Join}(B_0[a_0], B_1[a_0], L'[a_0])$. *This is the step the paper's proof misses*: the functions are compared at the join of their types, not at a common type.

Lemma 8 (diagonal). *If $\text{Join}_\Gamma(T_0, T_1, L), \Gamma \vdash T_0 = T_1, t_i : T_i$ and $\text{coe}_0(t_0) = \text{coe}_1(t_1) : L$, then $\Gamma \vdash t_0 = t_1 : T_0$.*

By induction on the join. In the universe case $k = m$ by injectivity of universes, so $L = U_{\max(k,k)} = U_k$. No lowering of a conversion from a larger universe is ever needed: this is why the level of a universe join is exactly $\max(k, m)$. In the Π case, $B_0 = B_1$ by Π -injectivity. Applying both λ 's to v_0 (after weakening) and using β , the coerced bodies are equal; then the induction hypothesis applies, and η concludes.

Theorem 9 (4.18, no normalisation). *If $\Gamma \vdash u_0 : A_0, \Gamma \vdash u_1 : A_1$, $\text{strip}(u_0) = \text{strip}(u_1)$ and $\Gamma \vdash A_0 = A_1$, then $\Gamma \vdash u_0 = u_1 : A_0$ (*term-uniq-conv-NF*); similarly for types (*type-uniq-NF*), and Lemma 4.18 (*term-uniq-NF*). Consequently Theorem 4.19 (*SectionR.agda*) holds without normalisation as well.*

`StripUniqNFTest.agda` checks the counterexample of §9: $\vdash u_0 = u_1 : U_2$ follows from the theorem.

Remark 10. η is used essentially: it is what makes coercions injective. The argument is a structural induction producing derivations from derivations; it uses no logical relation and no normalisation. The three properties it takes as input are proved here from the domain model; the paper assumes them (from normalisation of the GAT, or from the finite-domain model).

7 The case $U : U$ (ERTUU/)

ERTUU/ is the type-in-type version: one universe, $U : U$, no levels and no cumulativity. It contains §4.2 ($T_{\text{arski}} \Leftrightarrow T_R$) and now also §4.3 ($T_R \Leftrightarrow T_P$). Everything is checked with `--safe (ERTUU/All.agda, 72 modules, about 13 s from scratch)`.

Why this case is interesting. T_P with $U : U$ does *not* normalise (Girard's paradox), so the route through β -normal forms (§8) is impossible. The only non-syntactic ingredient, Π -injectivity of T_R , comes from the domain model, and its adequacy proof does not need normalisation.

Theorem 11 (ERTUU/PiInjectivityR.agda). *In T_R with $U : U$: if $\Gamma \vdash \Pi(C_0, B_0) = \Pi(C_1, B_1)$ then $\Gamma \vdash C_0 = C_1$ and $\Gamma.C_0 \vdash B_0 = B_1$.*

The model (`Dom/`, `Model/`) is the finite-element model of `ERT/`, with levelled universe codes. U is interpreted as the level-0 universe code. This is sound because the model's membership relation ignores levels: soundness (`Model/RussellSound`) goes through with $U : U$ interpreted as " $U_0 : U_0$ ". The validity relation and the adequacy driver (`Valid/`, `Adeq/`) are those of `ERT/` with the universe levels removed from the syntax. The only change needed by hand is that U now evaluates to the level-0 code only (`Adeq/Stmt.evU`).

Theorem 12 (4.15, 4.17/4.18, 4.19 for $U : U$). *StripDeriv: T_R derivations strip to T_P derivations. StripUniq: if $\Gamma \vdash u_0 : T_0, \Gamma \vdash u_1 : T_1$ and $\text{strip}(u_0) = \text{strip}(u_1)$, then $\Gamma \vdash T_0 = T_1$ and $\Gamma \vdash u_0 = u_1 : T_0$; no hypothesis on the types is needed. SectionR: every T_P judgement lifts into every T_R context over its context, uniquely up to conversion.*

Without cumulativity, types are unique up to conversion, and the paper's proof of Lemma 4.17 (induction on the term, with Π -injectivity in the application case) works as written. The joins and coercions of §6 are not needed: the counterexample of §9 relies on cumulativity. Neither injectivity of universes nor no-confusion is used.

The T_R rules of ERTUU/ now carry the same presupposition premises as those of `ERT/` (§4); the §4.2 files (`EraseDeriv`, `Equivalence`) use the paper-form lemmas `mk-*`. The `CoverageNoExactSplit` warning in `ERTUU/Uniqueness.term-uniq` (§4.2) was harmless: a catch-all clause for a conversion on the right overlapped the left-hand conversion clause, so it failed to hold definitionally only when both sides were conversions. It is now split into one clause per left-hand head, and the whole of ERTUU/ checks with no warnings. (The analogous warning in `ERT/UniquenessTermPartial` is of the same benign kind and is still present.)

8 What is proved assuming weak β -normalisation (superseded)

WN_P : if $\Gamma \vdash M : A$ in T_P then $M \rightarrow_\beta^* N$ for some β -normal N

(β at all positions of T_P terms; `PReduction.agda`). It is a module parameter, not a postulate.

Theorem 13 (4.18, `StripUniq.agda`). Assume WN_P .

1. If $\Gamma \vdash A_0$ type, $\Gamma \vdash A_1$ type in T_R and $\text{strip}(A_0) = \text{strip}(A_1)$, then $\Gamma \vdash A_0 = A_1$ (*type-uniq-R*).
2. If $\Gamma \vdash u_0 : A_0$, $\Gamma \vdash u_1 : A_1$ and $\text{strip}(u_0) = \text{strip}(u_1)$ and $\Gamma \vdash A_0 = A_1$, then $\Gamma \vdash u_0 = u_1 : A_0$ (*term-uniq-conv-R*). In particular (Lemma 4.18) this holds if $\text{strip}(A_0) = \text{strip}(A_1)$ (*term-uniq-R*).

Proof (Streicher’s method, *Semantics of Type Theory*, Thm 4.12): normalise $\text{strip}(u_0)$, perform the same reduction sequence on u_0 and on u_1 (lifting lemma), use subject reduction, and conclude by the normal-form lemma. Lemma 4.17 and strip' are not needed.

Theorem 14 (4.19, `SectionR.agda`). Assume WN_P . Let $\Gamma \vdash J$ in T_P .

1. For every Γ' with $\vdash \Gamma'$ in T_R and $\text{strip}(\Gamma') = \Gamma$ there is a derivation $\Gamma' \vdash J'$ in T_R with $\text{strip}(J') = J$ (*lift-IsType*, *lift-HasType*, *lift-ConvTy*, *lift-ConvTm*); and such a Γ' exists (*lift-WfCtx*).
2. Two such J' in the same Γ' are equal up to conversion (*section-uniq-Tm*, *section-uniq-Ty*, i.e. Theorem 4.18).

The lift is by structural induction on the T_P derivation; annotations are read off the premises, and premises that share a term or a type are reconciled by Theorem 4.18.

9 The gap in the paper’s proof of Lemma 4.17

The statement (Lemma 4.17 of `sterbac1.pdf`, Lemma 30 of `sterbac3.pdf`). With $\text{strip}'(\Pi(A, B)) = \text{strip}'(B)$ and $\text{strip}' = \text{strip}$ otherwise: (i) if $\Gamma \vdash A$ type, $\Gamma \vdash B$ type and $\text{strip}(A) = \text{strip}(B)$ then $\Gamma \vdash A = B$; (ii) if $\Gamma \vdash u_0 : A_0$, $\Gamma \vdash u_1 : A_1$, $\text{strip}(u_0) = \text{strip}(u_1)$ and $\text{strip}'(A_0) = \text{strip}'(A_1)$ then $\Gamma \vdash u_0 = u_1 : A_0$ and $\Gamma \vdash A_0 = A_1$. The proof is “by mutual induction on A and u_0 ”.

The application case assumes that the last rule of the derivation of $\Gamma \vdash u_0 : A_0$ is the application rule, so that A_0 is $B_0[a_0]$, and derives $\text{strip}'(B_0) = \text{strip}'(B_1)$ from $\text{strip}'(B_0[a_0]) = \text{strip}'(B_1[a_1])$, then applies the induction hypothesis to the functions to get $\Gamma \vdash \Pi(C_0, B_0) = \Pi(C_1, B_1)$. In T_R the last rule may be *cumulativity*. Counterexample (`Gap417.agda`, checked in Agda), in the empty context:

$$\begin{aligned} f_0 &= \lambda(U_1, U_1, v_0) : \Pi(U_1, U_1), & f_1 &= \lambda(U_1, U_2, v_0) : \Pi(U_1, U_2) \quad (v_0 : U_1 \leq U_2), \\ u_0 &= \text{app}(U_1, U_1, f_0, U_0) : U_1 \leq U_2, & u_1 &= \text{app}(U_1, U_2, f_1, U_0) : U_2. \end{aligned}$$

Here $\text{strip}(u_0) = \text{strip}(u_1) = (\lambda U_1 v_0) U_0$ and $A_0 = A_1 = U_2$, so the hypotheses of (ii) hold. But $\text{strip}'(\Pi(U_1, U_1)) = U_1 \neq U_2 = \text{strip}'(\Pi(U_1, U_2))$, so the induction hypothesis does not apply to f_0, f_1 , and the conclusion the proof needs, $\vdash \Pi(U_1, U_1) = \Pi(U_1, U_2)$, is *false* (Π -injectivity and injectivity of universes; *no-Pi-conv*). The statement itself is not refuted: $\vdash u_0 = u_1 : U_2$ holds, both sides β -reducing to U_0 ($u_0 = u_1$).

Further weak steps (not formalised as counterexamples).

- “ $\text{strip}'(B_0[a_0]) = \text{strip}'(B_1[a_1])$ implies $\text{strip}'(B_0) = \text{strip}'(B_1)$ ” fails for open codomains, since substitution may identify different codomains ($B_0 = v_0$, $B_1 = U_1$, $a_0 = a_1 = U_1$).
- In the λ case the proof uses $\text{strip}'(\Pi(C_0, B_0)) = \text{strip}(B_0)$, i.e. it assumes A_0 is literally $\Pi(C_0, B_0)$; in T_R , A_0 is only convertible to it.
- The induction measure (“on A and u_0 ”) is not made precise, and the type part (i) inherits the problem: types may be β -redexes whose λ ’s codomain annotations differ by cumulativity.

Why Streicher’s argument does not transfer. In *Semantics of Type Theory*, Thm 4.11 (strip_1 , dropping the annotations of applications) is proved by induction on the number of applications, and its application case uses Thm 4.10, i.e. *uniqueness of types* (and injectivity of Π). T_R has no uniqueness of types because of cumulativity, and strip' does not repair this. Streicher’s Thm 4.12 (dropping also λ ’s annotation) proceeds instead through β -normal forms; this is the method used above, under WN_P . The paper notices the cumulativity phenomenon only for the further step of erasing λ ’s domain (Example 4.20 / 33), where it states that normalisation is needed. Note also that §4.2 (Tarski $\Leftrightarrow T_R$) is not affected: there the erasure keeps the annotations of `app`, and the formalised proof (`Uniqueness.agda`, `UniquenessTermPartial.agda`, `Equivalence.agda`) uses no normalisation.

Consequence. The gap is in the proof, not in the method: Lemma 4.18 and Theorem 4.19 hold with a purely syntactic proof (§6), provided the induction hypothesis compares terms at the *join* of their types. So the claim of `sterbac3.pdf` that the proof is syntactic, relative to injectivity and no-confusion, survives for §6 too.

Agda files

RussellTyping, RussellMeta, RussellMetaCong	T_R and its metatheory
PTyping, PReduction	T_P , β -reduction, normal forms, WN_P
Model/*, Dom/*, Valid/*, Adeq/*	domain model, soundness, validity, adequacy
PiInjectivityR, RussellInversion	Π -injectivity, universe injectivity, inversion
StripDeriv	Theorem 4.15
RussellStep, SubjectReductionR	lifting of reductions, subject reduction
StripUniqNF	coercions, joins, Theorem 4.18 (no normalisation)
SectionR	Theorem 4.19 (no normalisation)
StripUniq	superseded: Theorem 4.18 under WN_P
Gap417	the counterexample to the proof of Lemma 4.17
ERTUU/	$U : U$: §4.2 and §4.3 (<code>All.agda</code> , <code>--safe</code>)
